

Introduction to Linux Tools

Exploring process monitoring, /proc, virtual memory, disk I/O, and system calls

Name:	Bodike Chaithali	Student ID:	B24CS013
Date:	04-08-2026	Branch(CS E/DSAI):	CSE

Instructions

- Complete each exercise in order. Later exercises assume you're comfortable with the tools from earlier ones.
- Run every command yourself in a terminal — don't just read the questions. Screenshots or pasted terminal output are required as evidence for each numbered question.
- Answer directly in the blanks provided, or attach a typed appendix if you need more space.
- Where a question asks you to "explain," a one-sentence answer is not sufficient — justify your observation using what you know about the OS.
- Submit this document along with your terminal output / screenshots as a single PDF.

Exercise 1 — Monitoring a CPU-Bound Process

Compile the program `cpu.c` provided to you and execute it in the shell:

```
gcc cpu.c -o cpu./cpu
```

This program runs in an infinite loop without terminating. Open a second terminal, run the top command, and answer the following questions about the cpu process.

Question 1a: What is the PID of the process running the cpu command?

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ls
cpu cpu.c cpu-print.c disk1.c disk.c make-copies.sh memory1.c memory2.c OS_Lab_Assignment_1.docx
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -ef | grep cpu
root      20      2  0 17:51 ?        00:00:00 [cpuhp/0]
root      21      2  0 17:51 ?        00:00:00 [cpuhp/2]
root      27      2  0 17:51 ?        00:00:00 [cpuhp/4]
root      33      2  0 17:51 ?        00:00:00 [cpuhp/5]
root      39      2  0 17:51 ?        00:00:00 [cpuhp/6]
root      45      2  0 17:51 ?        00:00:00 [cpuhp/7]
root      51      2  0 17:51 ?        00:00:00 [cpuhp/1]
root      57      2  0 17:51 ?        00:00:00 [cpuhp/3]
root     5780    2  0 17:57 ?        00:00:00 [kworker/1:2-mm_percpu_wq]
root     8804    2  0 18:03 ?        00:00:00 [kworker/1:1-mm_percpu_wq]
bodike   9451   8270 99 18:06 pts/1    00:00:36 ./cpu
bodike   9620   9521  0 18:06 pts/0    00:00:00 grep cpu
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

Question 1b: How much CPU and memory does this process consume?

```
top - 18:15:27 up 23 min, 1 user, load average: 1.45, 1.36, 1.08
Tasks: 305 total, 2 running, 303 sleeping, 0 stopped, 0 zombie
%Cpu(s): 12.7 us, 0.1 sy, 0.0 ni, 87.2 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7627.3 total, 458.4 free, 2051.3 used, 5117.6 buff/cache
MiB Swap: 2048.0 total, 1199.7 free, 848.2 used. 4903.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
9451	bodike	20	0	2644	1144	1052	R	100.0	0.0	9:04.22	cpu
2965	mongodb	20	0	577872	50688	26612	S	0.7	0.6	0:09.45	mongod
35	root	rt	0	0	0	0	S	0.3	0.0	0:00.35	migration/s
902	mysql	20	0	2374684	35048	11700	S	0.3	0.4	0:12.34	mysqld
3566	999	20	0	3784676	125252	22536	S	0.3	1.6	0:13.33	mongod
3750	root	20	0	11.3g	99440	50320	S	0.3	1.3	0:05.14	node
11769	root	20	0	0	0	0	I	0.3	0.0	0:00.13	kworker/u16:0-writeback
12126	bodike	20	0	13592	4680	3688	R	0.3	0.1	0:00.01	top
1	root	20	0	167144	12456	8400	S	0.0	0.2	0:01.75	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd

Question 1c: What is the current state of the process? For example, is it running, in a blocked state, or a zombie state?

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -o pid,state,comm -p 9451
PID S COMMAND
9451 R cpu
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

Exercise 2 — Parent–Child Process Chains

This exercise explores how the Linux shell (e.g., the bash shell) runs user commands by spawning new child processes to execute them.

Compile the program `cpu-print.c` provided to you and execute it in a bash shell (or any other shell of your choice):

```
gcc cpu-print.c -o cpu-print./cpu-print
```

Question 2a: Find the PID of the parent of the `cpu-print` process, i.e., the shell process.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -ef | grep cpu-print
bodike    15358   14995  56 19:55 pts/1    00:00:05 ./cpu-print
bodike    15397   14990   0 19:55 pts/0    00:00:00 grep cpu-print
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -o pid,ppid,comm -p 15358
PID      PPID COMMAND
15358    14995 cpu-print
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -o pid,ppid,comm -p 15397
PID      PPID COMMAND
15397    15358 grep
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

Question 2b: Find the PIDs of all the ancestors of the cpu-print process, going back at least 5 generations (or until you reach the init process). List each PID and the command it corresponds to.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ pstree -sp 15358
systemd(1)─systemd(1764)─gnome-terminal-(14972)─bash(14995)─cpu-print(15358)
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

Question 2c: Using the pstree command, print the process tree of the shell process that is running your cpu-print program. Attach the output as evidence.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ pstree -p $(ps -o ppid= -p $(pgrep cpu-print))
bash(5363)─cpu-print(23064)
           └─sleep(6617)
```

Question 2d: Stop the execution of the cpu-print executable. Run a long background command, e.g., sleep 10000 &, in your shell. Then restart the cpu-print process in the foreground again. Visualize the process tree of your shell once more — how, if at all, did it change?

Earlier the shell had only cpu-print as a child process. After running sleep in the background, the shell has two child processes: sleep (background) and cpu-print (foreground). The process tree therefore contains an additional child process.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ pstree -p 5363
bash(5363)─cpu-print(24020)
           └─sleep(6617)
              └─sleep(24019)
```

Exercise 3 — Shell Builtins vs. Executables

When you type a command into the shell, the shell does one of two things. For some commands, executables that perform that functionality already come with your Linux kernel installation, and the shell simply invokes the executable. For other commands, the shell implements the command itself within its own code.

Consider the following commands you can type into the bash shell: cd, ls, ps, sleep, history. For each, determine whether it exists as an executable in the Linux filesystem or is implemented by the bash shell itself. You may use the which command to help.

Command	Builtin or Executable?	Pathname (if executable)
---------	------------------------	--------------------------

cd		
ls		
ps		
sleep		
history		

Explain briefly why a shell would implement some commands itself rather than always invoking a separate executable.

The shell implements commands such as **cd** and **history** as built-in commands because they need to modify or access the shell's own environment. For example, **cd** changes the current working directory of the shell itself, and **history** accesses the shell's command history. If these commands were separate executables, they would run in child processes and would not be able to modify the parent shell's state. Commands such as **ls**, **ps**, and **sleep** are independent programs stored in the file system, so the shell simply executes them when requested.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ which cd
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ type cd
cd is a shell builtin
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ which ls
/usr/bin/ls
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ type ls
ls is hashed (/usr/bin/ls)
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ which ps
/usr/bin/ps
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ type ps
ps is /usr/bin/ps
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ which sleep
/usr/bin/sleep
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ type sleep
sleep is /usr/bin/sleep
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ which history
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ type history
history is a shell builtin
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ █
```

Exercise 4 — Virtual vs. Physical Memory

Consider the two programs `memory1.c` and `memory2.c` provided to you. Compile and run them one after the other:

```
gcc memory1.c -o memory1 && ./memory1 gcc memory2.c -o memory2 && ./memory2
```

Both programs allocate a large array in memory. One of them accesses the array; the other does not. Both pause before exiting so you can inspect their memory usage with the `ps` command. In particular, `ps` reports the total “virtual” memory size of the process (the memory the process thinks it has) and how much of that is actually physically resident in RAM.

Question 4a: While each program is paused, use `ps` to report the virtual memory size (VSZ) and resident memory size (RSS) of each process.

memory1: VSZ = 6560 KB, RSS = 5492 KB

memory2: VSZ = 6564 KB, RSS = 5492 KB

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ pgrep memory1
7826
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -o pid,vsz,rss,comm -p 7826
  PID   VSZ   RSS COMMAND
  7826  6560  5492 memory1
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ pgrep memory2
8672
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ ps -o pid,vsz,rss,comm -p 8672
  PID   VSZ   RSS COMMAND
  8672  6564  5492 memory2
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

Question 4b: Compare the virtual and physical memory usage of both programs. What differs, and why?

Both programs have nearly the same virtual memory size (VSZ) and resident memory size (RSS). In this experiment, there is no significant difference between their memory usage. This indicates that both programs occupied almost the same amount of virtual and physical memory while they were running. The operating system allocated a similar amount of physical memory for both processes during execution.

Question 4c: Inspect the source code of both programs. Explain your observations in terms of what each program actually does with its allocated array.

After inspecting the source code, both programs allocate a large array in memory. Normally, a program that accesses its allocated memory causes the operating system to load the required memory pages into RAM through demand paging, while a program that does not access the memory may use less physical memory. However, in this experiment, both programs showed almost identical VSZ and RSS values, indicating that the operating system allocated a similar amount of physical memory for both during execution.

Exercise 5 — Disk I/O

Compile and run the programs `disk.c` and `disk1.c` provided to you. These programs read a large number of files from disk, which you must first create as follows:

- Create a folder named `disk-files` and place the file `foo.pdf` inside it.
- Use the script `make-copies.sh` to make 5000 copies of `foo.pdf` in that folder, each with a different filename.

Ensure `disk-files` is located on your local disk, not a network-mounted directory (e.g., NFS) — if it is, adjust the file path in the provided code so it reads from local disk.

Run the disk programs one after the other, and use a tool such as `iostat` to measure disk utilization while each program runs.

disk.c 0.20 %

The `disk1.c` program repeatedly reads the same file (`f000.pdf`). Normally, repeated access to the same file allows the Linux page cache to serve subsequent reads from memory instead of the physical disk. In my experiment,

`iostat` reported approximately 25 reads/sec, 2540 kB/sec, and 1.00% disk utilization. Although the measured utilization was higher than expected, this can vary because of operating system caching, background disk activity, and the fact that the test files were empty. The experiment still demonstrates how the operating system manages file I/O and caching.

Device	r/s	kB/s	rrqm/s	%rrqm	r_wait	rareq-sz	w/s	kB/s	wrqm/s	%wrqm	w_wait	wareq-sz	d/s	kB/s	drqm/s	%drqm	d_wait	dareq-sz	f/s	f_wait	aqu-sz	%util
loop0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop12	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop13	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop14	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop15	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop16	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop17	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop18	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop19	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop21	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop22	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop24	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop25	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop26	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop27	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop28	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop29	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop3	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop30	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop4	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop5	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop6	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop7	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop8	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop9	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
nvme0n1	0.00	0.00	0.00	0.00	0.00	0.00	25.00	2540.00	33.00	56.90	2.10	101.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	2.00	0.00	1.00

Exercise 6 — Tracing System Calls with `strace`

User programs make system calls — requests to the OS kernel, similar to function calls but more involved — throughout their execution. The `strace` tool lets you trace the system calls a running executable invokes.

Choose any executable from the exercises above, or a standard Linux command such as `ls`, and run it under `strace`:

```
strace ls
```

Question 6a: How many system calls in total does the program make?

The `ls` command made a total of **80 system calls** during execution. Five of these returned expected errors while checking for optional files or resources.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$ strace -c ls
cpu    cpu-print  disk  disk1.c  disk-files  make-copies.sh  memory1.c  memory2.c  OS_Lab_Assignment_1.odt
cpu.c  cpu-print.c  disk1  disk.c  foo.pdf  memory1  memory2  OS_Lab_Assignment_1.docx
% time  seconds  usecs/call  calls  errors  syscall
-----
28.14   0.000213      11      18      mmap
12.81   0.000097      13       7      mprotect
12.15   0.000092      13       7      openat
 6.61   0.000050      10       5      read
 5.68   0.000043       4       9      close
 5.02   0.000038      38       1      munmap
 4.89   0.000037      18       2      getdents64
 4.62   0.000035       4       8      newfstatat
 4.36   0.000033      11       3      brk
 3.57   0.000027      13       2      write
 3.17   0.000024      12       2      2 statfs
 2.11   0.000016       4       4      pread64
 1.59   0.000012       6       2      ioctl
 1.32   0.000010       5       2      2 access
 0.92   0.000007       7       1      set_robust_list
 0.79   0.000006       6       1      getrandom
 0.66   0.000005       5       1      prlimit64
 0.53   0.000004       2       2      1 arch_prctl
 0.53   0.000004       4       1      set_tid_address
 0.53   0.000004       4       1      rseq
 0.00   0.000000       0       1      execve
-----
100.00  0.000757       9      80      5 total
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/Lab Assignment 1/LAB-1$
```

Question 6b: Look up the list of system calls supported by your operating system. Identify at least three system calls that appear in your strace output, and briefly describe what each one does.

- **execve()** – Executes the `ls` program by replacing the current process image with the executable.
- **openat()** – Opens files or directories, such as shared libraries and the current working directory.
- **read()** – Reads data from opened files into memory.
- **write()** – Writes the output of the `ls` command to the terminal.
- **getdents64()** – Reads directory entries so that `ls` can list all files and folders.

Submission Checklist

- Answers to all questions in Exercises 1–6
- Terminal output / screenshots supporting each answer
- Completed as a single PDF, named LASTNAME_FIRSTNAME_lab.pdf